PROJECT REPORT

(CSA2001) Fundamentals in AL & ML

Slot-C11+C12+C13, BL2025260500811

Winter Semester: 2025-2026

Name of Student: Akshat Rajput

Registration Number: 25BCE10874

Guided By- Dr. Monica Sankat

Introduction-

"Face Mask Detection System" is a python - based computer vision application that detects whether a person in an image is wearing masks or not. As during Covid-19 pandemic, wearing face masks became a critical public health measure and it is still a critical health measure in hospitals and monitoring every individual whether he is wearing masks or not is very difficult.

This project implements a deep learning-based solution using Custom CNN Model built from scratch in Pytorch, combined with OpenCV, for face detection and displaying the result.

The system uses: -

CNN- CNN known as Convolutional Neutral Network it is a deep learning algorithm designed to analyse visual data like images and video mimicking the biological process of the human visual cortex. It automatically learns hierarchical patterns ranging from simple edges to complex shapes making it standard for image classification, object detection and segmentation tasks.

PyTorch- It's an open-source machine learning framework used for building and training deep neural networks.

OpenCV- It's a massive, open-source library featuring over 2500 algorithms for real-time image processing, computer vision, and machine learning.

Feature used in this Project:

- Haar cascade classifier- A pre-trained model for detecting frontal by scanning the image.
- Image processing- reading, resizing, grayscale conversion of image.
- Visual Output- Drawing bounding boxes and text labels on detected faces.

Tkinter- Tkinter is Python's standard GUI library. It provides a file dialog window for the user to select an image for prediction, replacing manual path entry.

Numpy- Numpy is used for numerical operations on multi-dimensional arrays. In this project, it handles image data as array and normalizes pixel values to the range.

Mathplotlib-Matplotlib is used to generate training accuracy/loss curves and the confusion matrix plot.

Scikit-learn- Scikit-learn provides machine learning utilities:

Train/Test Split- Splitting the dataset into 80% training and 20% testing.

Classification Report- Generating precision, recall, and F1-score.

Confusion Matrix-Visualizing model performance across classes.

Problem Statement and Objective-

Manual enforcement of mask compliance is slow, inconsistent, and not scalable. A single guard at a gate cannot simultaneously check hundreds of people. An automated system that can detect mask compliance using image analysis can.

This problem sits on the intersection of computer vision and deep learning.

Non- Functional Requirements-

1- Reduce human effort- No need for dedicated staff at every entry point.
2- Provide instant feedback- Visual indicators on the output image.
3- Scale easily- Deploy on any device that can run python.
4- Adapt to similar problems- The same pipeline can be repurposed for detecting helmets, safety gear, or other compliance checks.

Objectives-

1. Accept an image as input via file dialog.
2. Detect all frontal faces in the image using Haar cascade.
3. Classify each detected face as Mask or No Mask.
4. Display the result with color-coded bounding boxes.

System Architecture-

The program uses this architect to produce output: -

- **Data Layer-** The dataset is organized into two folders ("with mask" or "without mask") inside the data directory. The trained model weights are persisted as "mask detector.pth"
- **Presentation Layer-**
  - Program ask input to what user like do he want to test image or exit from program.

```
======== RESTART: D:\Winter S

======MASK DETECTOR======
1-Test Image
2-Exit
Enter your choice: 1

======MASK DETECTOR======
1-Test Image
2-Exit
Enter your choice: |
```

- **Processing Layer-**

```
Image Input
      |
      ▼
┌──────────────────┐
│  Face Detection  │  ← Haar Cascade (OpenCV)
└──────────────────┘
      | (cropped face regions)
      ▼
┌──────────────────┐
│  Preprocessing   │  ← Resize to 64x64, normalize [0,1]
└──────────────────┘
      |
      ▼
┌──────────────────┐
│  CNN Classification │  ← Custom PyTorch model
│  (Mask / No Mask)   │
└──────────────────┘
      |
      ▼
┌──────────────────┐
│  Visual Output   │  ← Bounding box + label on image
└──────────────────┘
```

CNN Model Architecture-

Layer	Type	Output Shape
1	Conv2D (3→32, 3x3) + ReLU	32 x 62 x 62
2	MaxPool2D (2x2)	32 x 31 x 31
3	Conv2D (32→64, 3x3) + ReLU	64 x 29 x 29
4	MaxPool2D (2x2)	64 x 14 x 14
5	Conv2D (64→128, 3x3) + ReLU	128 x 12 x 12
6	MaxPool2D (2x2)	128 x 6 x 6
7	Flatten	4608
8	Linear (4608→128) + ReLU	128
9	Dropout (0.5)	128
10	Linear (128→2)	2

Key Design Decisions: -

- Three Convolutional Blocks- Sufficient depth to extract edges → textures → mask patterns without overcomplicating the model.
- Dropout (0.5)- Prevents Overfitting on the relativity small dataset.
- No Batch Normalization- Keep the architecture simple, can be added in future iterations.

Implementation Details-

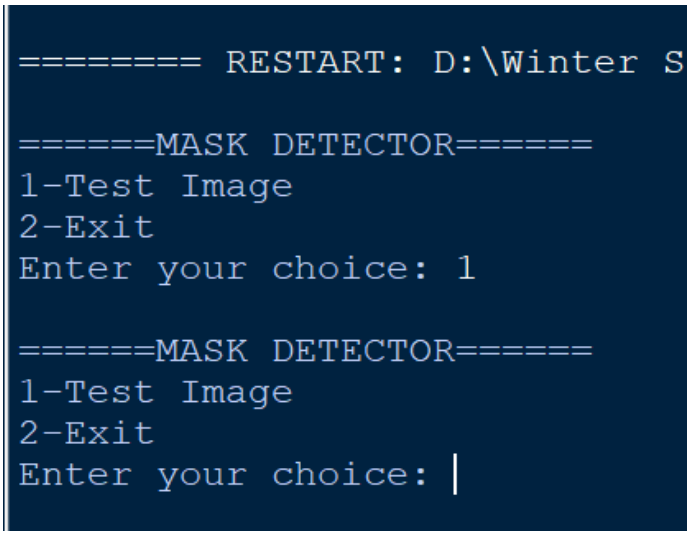
- **Library-** Tkinter, PyTorch, OpenCV, NumPy, Matplotlib, Scikit-Learn
- **Training Configuration-**
    - Loss Function – CrossEntropyloss
    - Optimizer- Adam (lr=0.001)
    - Batch Size- 32
    - Epochs- 10
    - Train / Test Split- 80% / 20%
    - Input Image Size- 64 x 64 pixels
    - Device- CUDA if GPU available, else CPU

File Structure-

```
--Face Mask Detection System/
        |
        --data/
            |-with mask/  (Training image of faces with mask)
            |-without mask/  (Training image of faces without mask)
        --train.py  (Model training, Model evaluation)
        --predict.py  (Image based mask detection inference)
        --Project Report.pdf  (Report of this Project)
        --Readme.md  (Project Overview)
        --Requirements.txt  (Python dependencies)
```

Screenshots / Results-

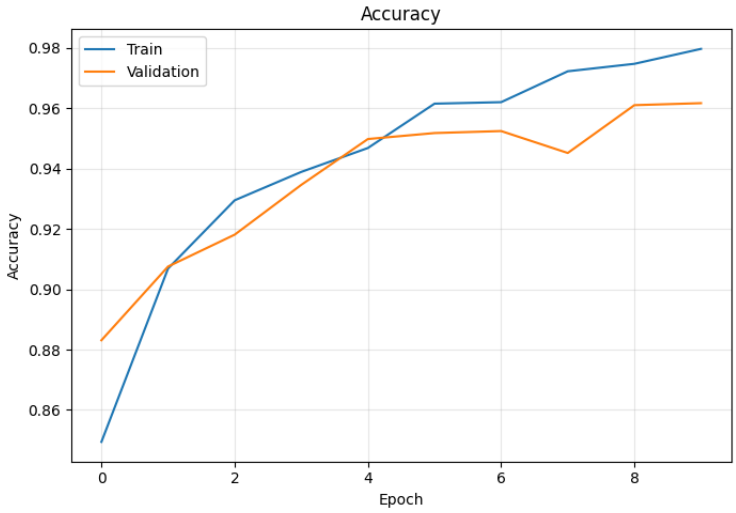
- Input Screen-

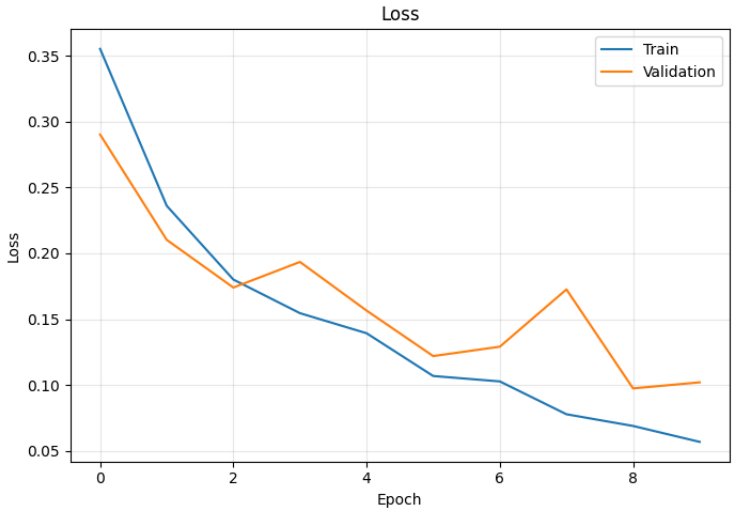


- Training Output-

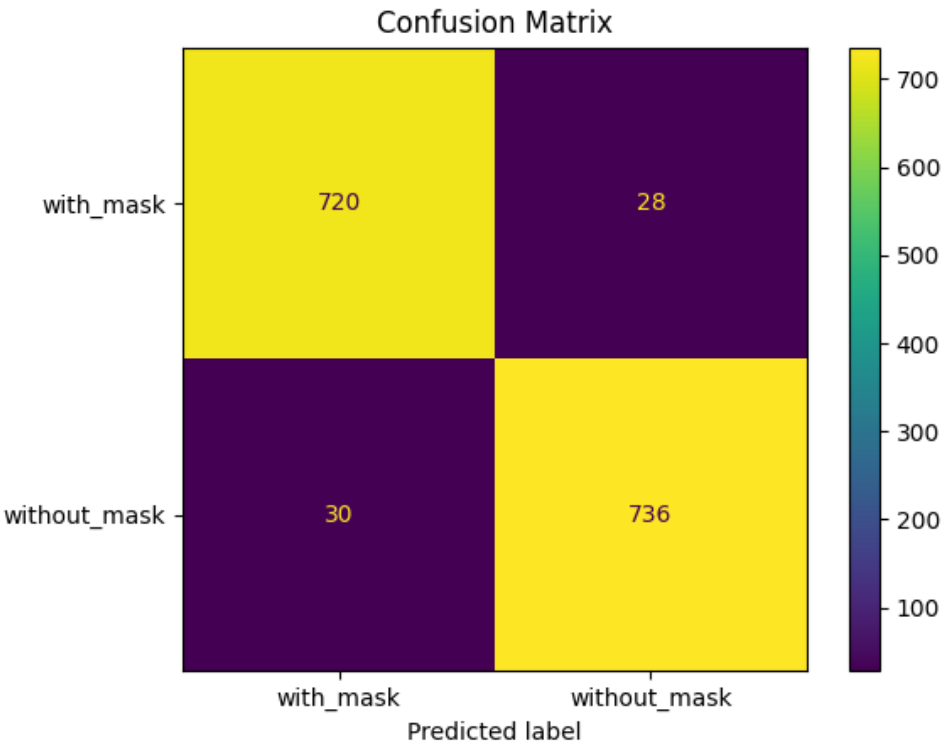
```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (A
Enter "help" below or click "Help" above for more information.

========= RESTART: D:\Winter Semester(2026)\AI ML\AIML Project\train.py ========
Using: cpu
Loading with_mask......
Loading without_mask......
Total images: 7570
Epoch 1: Train Acc=0.8494, Val Acc=0.8831, Train Loss=0.3554, Val Loss=0.2903
Epoch 2: Train Acc=0.9069, Val Acc=0.9075, Train Loss=0.2361, Val Loss=0.2103
Epoch 3: Train Acc=0.9295, Val Acc=0.9181, Train Loss=0.1801, Val Loss=0.1740
Epoch 4: Train Acc=0.9389, Val Acc=0.9346, Train Loss=0.1545, Val Loss=0.1935
Epoch 5: Train Acc=0.9468, Val Acc=0.9498, Train Loss=0.1393, Val Loss=0.1565
Epoch 6: Train Acc=0.9615, Val Acc=0.9518, Train Loss=0.1068, Val Loss=0.1219
Epoch 7: Train Acc=0.9620, Val Acc=0.9524, Train Loss=0.1027, Val Loss=0.1291
Epoch 8: Train Acc=0.9723, Val Acc=0.9452, Train Loss=0.0777, Val Loss=0.1726
Epoch 9: Train Acc=0.9747, Val Acc=0.9610, Train Loss=0.0688, Val Loss=0.0974
Epoch 10: Train Acc=0.9797, Val Acc=0.9617, Train Loss=0.0568, Val Loss=0.1020

Accuracy: 0.9617

Classification Report:
              precision    recall  f1-score   support

   with_mask       0.96      0.96      0.96       748
without_mask       0.96      0.96      0.96       766

    accuracy                           0.96      1514
   macro avg       0.96      0.96      0.96      1514
weighted avg       0.96      0.96      0.96      1514

Model saved!!
Training plots saved as training_history.png
```

- Accuracy and Loss Curves-



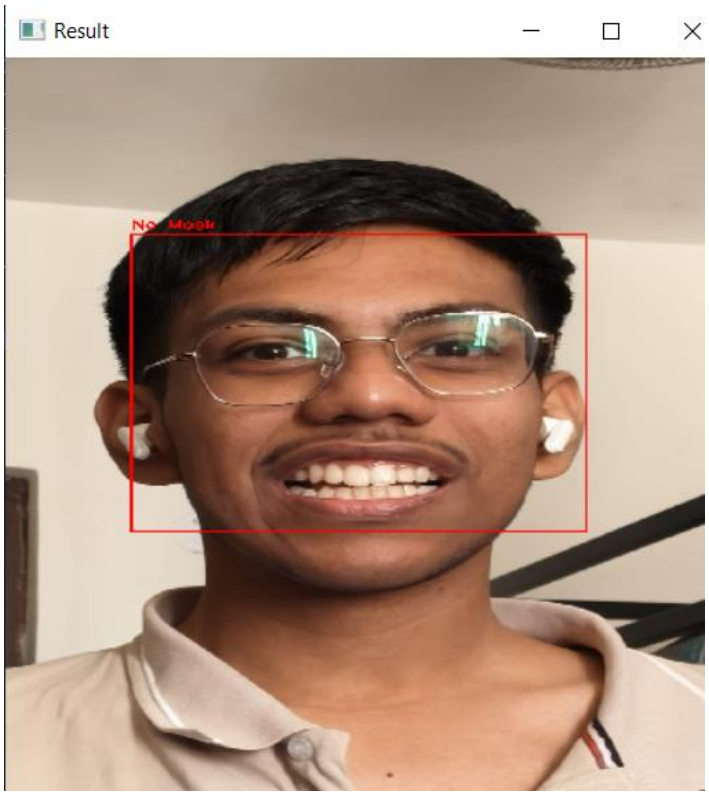


- Confusion Matrix-The Confusion Matrix displays the model's classification performance on the test set

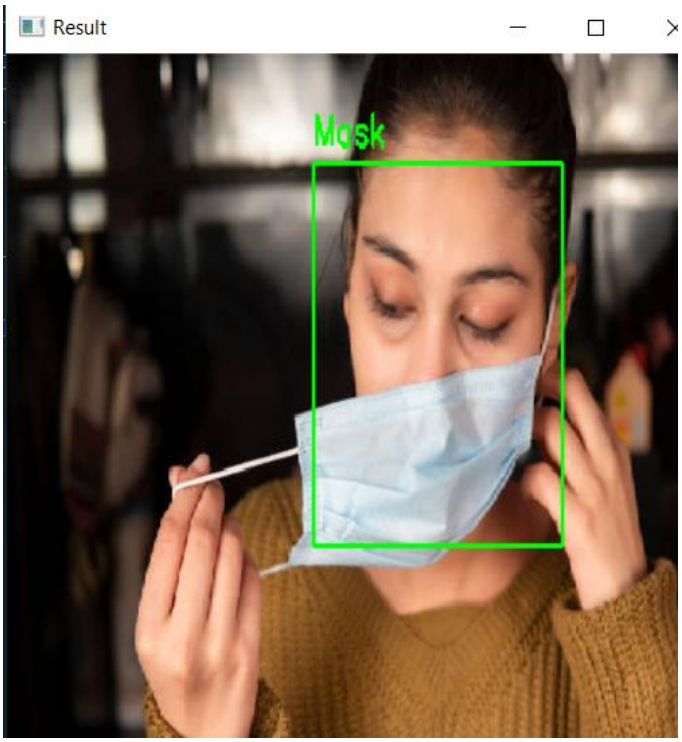


Testing Approach-

Test-1 (Without Mask)- When user Select image without Mask program identify and put face in red square.

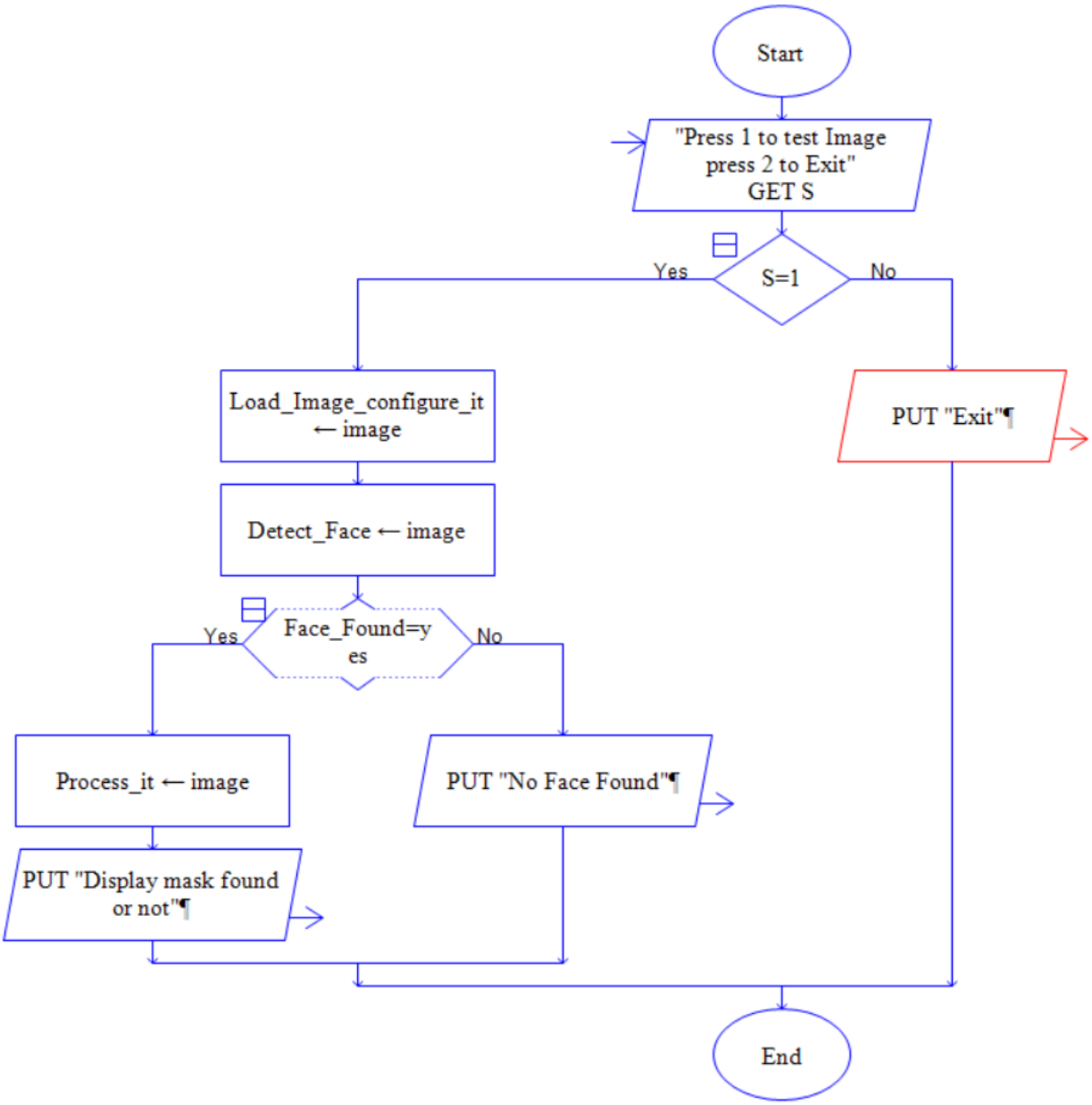


Test-2 (With Mask)- When user Select image with Mask program identify and put face in green square.



Program Workflow-

- **Flowchart-**



(Program Workflow using Flowchart)

- **Algorithm-**

Step 1: Load the dataset containing images of faces with and without masks.

Step 2: Preprocess images: resize to 64×64, normalize pixel values to [0, 1].

Step 3: Split data into 80% training and 20% testing sets.

Step 4: Train the CNN model for 10 epochs using Adam optimizer.

Step 5: Save the trained model weights as “mask_detector.pth”.

Step 6: For prediction: accept an image via file dialog.

Step 7: Detect faces using OpenCV's Haar Cascade classifier.

Step 8: For each detected face, crop, preprocess, and pass through the CNN.

Step 9: The model outputs a prediction: Mask (green box) or No Mask (red box).

Step 10: Display the annotated result image.

Challenges Faced-

- Keeping Model in Sync.
- Finding Best Dataset to Train Model.
- Getting perfect configuration of CNN so that Model is able to Handel all kind of image and print perfect output as fast as possible.
- Finding compatible model.

Learnings & Key Takeaways-

- **CNN Architecture-** Build a CNN model from scratch gave me deep insides about the model and its functioning.
- **PyTorch-** Learning how to use PyTorch.
- **Tensor shapes-** The most common bugs were shape mismatches. Understanding permute(), unsqueeze(), and channel-first vs. channel-last conventions was essential.

Future Enhancements-

1. **GUI Interface:** Replace the console input with a tkinter or PyQt form for a more modern look.
2. **DNN Based face detector-** To improve the accuracy of the Model.
3. **Web deployment-** To ensure the easy access without running python scripts directly.
4. **Live Detection-** Running Live detection and integrating it with hardware to make consumer level version of it.

Steps to Run Project-

1. Install Python from Official source https://www.python.org/downloads/
2. Install Required packages –

   pip install torch torchvision opencv-python numpy scikit-learn matplotlib

3. Download Training dataset.
4. First run train.py file to train the model.
5. Then you can run the predict.py to use the feature of predicting the image.

References-

1. PyTorch Documentation — https://pytorch.org/docs/stable/

2. OpenCV Documentation — https://docs.opencv.org/

3. Haar Cascade Classifier-https://docs.opencv.org/4.x/db/d28/tutorial_cascade_classifier.html

4. Scikit-learn Metrics — https://scikit-learn.org/stable/modules/model_evaluation.html

5. Face Mask Dataset — https://www.kaggle.com/datasets/omkargurav/face-mask-dataset

6. NumPy Documentation — https://numpy.org/doc/

7. Matplotlib Documentation — https://matplotlib.org/stable/contents.html

8. Dataset- https://www.kaggle.com/datasets/omkargurav/face-mask-dataset